

MOSIP Decode 2026 — Problem Statement 1

Automated Conformance Testing for Inji Certify & Inji Verify

Findings Log and Open Questions

Shardul Chogale

Mukta Varak

Document started: 20 September 2026

Last updated: 29 September 2026

Abstract

This document records, day by day, the conformance testing work carried out against the Inji stack as part of MOSIP Decode Problem Statement 1. It captures what was run, what failed, the evidence for each failure, and the questions we need answered by MOSIP mentors. Every finding is tagged as **CONFIRMED** or **UNVERIFIED**; we do not report a defect upstream until it is reproducible and the root cause is understood.

Contents

1	Scope and System Under Test	2
2	Day Log	3
2.1	Day 1 — Saturday, 20 September 2026	3
2.2	Day 2 — Tuesday, 23 September 2026	4
2.3	Day 3 — Wednesday, 24 September 2026	4
2.4	Day 4 — Monday, 29 September 2026	6
3	Findings Catalogue	9
	F-01 — Nonce has insufficient entropy	9
	F-02 — Nonce contains non-URL-safe characters	10
	F-03 — No DCQL support; test plan cannot proceed	10
	F-04 — Uses superseded <code>presentation_definition</code>	10
	F-05 — <code>client_metadata</code> absent in inline mode	10
	F-06 — No <code>direct_post.jwt</code> ; HAIP certification impossible	11
	F-07 — DID document verification method identifier inconsistency	11
	F-08 — No mdoc / ISO 18013-5 support	11
	F-09 — Client identifier prefix has no overlap with 1.0 Final	12
4	Open Questions for Mentors	13
5	Reproduction	15

1 Scope and System Under Test

Problem statement

Build an automated harness that spins up Inji Certify and Inji Verify, drives the OpenID Foundation conformance suite's REST API programmatically to run the issuer and verifier test plans, and feeds the results into each module's existing `api-testrig`, producing benchmark-gated reports with CI/CD integration.

Components under test

Component		Version	Notes
Inji Verify (service)		0.18.2	<code>injistack/inji-verify-service</code>
Inji Verify (UI)		0.18.2	<code>injistack/inji-verify-ui</code> , amd64 under emulation on arm64
Inji Certify		0.14.0	Not yet exercised (deferred, see Plan)
OpenID Suite	Conformance	5.3.1 (rev 440eec8)	Local Docker deployment, prebuilt images

Test environment

- Host: Apple Silicon (arm64) macOS, Docker Desktop.
- Conformance suite reachable at `https://localhost.emobix.co.uk:8443` (self-signed certificate; hostname resolves publicly to 127.0.0.1).
- Inji Verify exposed publicly via an ngrok HTTPS tunnel. A public hostname is mandatory because Verify identifies itself with a `did:web` DID, which is resolved over public HTTPS.
- The conformance suite's REST API required no authentication token in the local `dev` profile, consistent with the problem statement.

Evidence handling

Conformance suite logs are exported as signed JSON (`.json` plus `.json.sig`) and stored under `logs/<date>/`. Findings below cite the exact log line (`src` field) and the OID4VP specification section reported by the suite.

2 Day Log

2.1 Day 1 — Saturday, 20 September 2026

Objective

Stand up the conformance suite and one MOSIP component, and complete one manual conformance test end to end to establish a baseline.

Decisions taken

1. **Inji Verify before Inji Certify.** Verify’s Docker Compose brings up three containers with no external dependencies. Certify’s `docker-compose-injistack` requires a PKCS12 keystore obtained through an onboarding process, a public DID endpoint, an external authorisation server, and partner API keys. The verifier test plan also has 12 modules against the issuer plan’s 61. Verify is the cheaper path to a first result, and everything learned transfers.
2. **Non-HAIP plan first.** The HAIP verifier plan offers only `direct_post.jwt` as a response mode. Inji Verify hardcodes `direct_post` (see F-06), so no HAIP test can complete. We ran *OpenID for Verifiable Presentations 1.0 Final: Test a verifier — alpha tests* instead, which offers plain `direct_post`.
3. **Credential format `sd_jwt_vc`.** The plan offers only `sd_jwt_vc` and `iso_md1`. Verify has no mdoc support whatsoever (F-08), making `iso_md1` unusable.

Test plan configuration

- Plan: `oid4vp-1final-verifier-test-plan`, plan ID `1N7C4DH1HvYmc`
- Variant: `credential_format=sd_jwt_vc,client_id_prefix=redirect_uri,request_method=url_query,vp_profile=plain_vp,response_mode=direct_post`
- Alias: `injiverify-shardul`

Runs executed

Test ID	Outcome	Notes
<code>dgeeosZpIfDK15</code>	FAILED / INTERRUPTED	First attempt. Request object had expired (5-minute window). Discarded as an artefact of manual timing, not a defect.
<code>dMg4B5whxWgShFA</code>	FAILED / INTERRUPTED	Fresh request (2m05s old). Suite never dereferenced <code>request_uri</code> because the <code>url_query</code> variant expects inline parameters. Root cause understood; see F-09.
<code>MtzipWoD2dVgtiFO</code>	FAILED / INTERRUPTED	After switching the credential’s <code>clientIdScheme</code> to <code>pre_registered</code> . Reached 20+ checks; produced the substantive findings below. This is the Day 1 baseline run.

Configuration change made

In `inji-verify/docker-compose/config/config.json`, the credential *Mock Identity (SD JWT)* was changed from `"clientIdScheme":"did"` to `"clientIdScheme":"pre_registered"`. Per Inji Verify’s own README, `did` delivers the authorisation request *by reference* via `request_uri`, whereas `pre_registered` delivers it *inline*. Only the inline form matches the `request_method=url_query` variant. Backup retained at `config.json.bak`.

Result of the baseline run (MtzpWoD2dVgtic0)

Approximately fourteen checks passed, including nonce extraction, response type, response mode, absence of `client_id_scheme`, absence of `scope`, absence of `redirect_uri`, response URI validity, and the outer parameter allow-list. Three checks failed and three raised warnings. The test terminated at `ExtractDCQLQueryFromAuthorizationRequest`.

Status at end of Day 1

Test 01 (`happy-flow`) complete and documented. Tests 02–11 deferred.

2.2 Day 2 — Tuesday, 23 September 2026

Calendar note

The work recorded above took place on 20 September, *three days before MOSIP Decode officially began*. The event kicked off on 23 September; the planned kickoff seminar and mentor AMA session did not take place. No work was carried out between 20 and 23 September, so everything listed as “deferred to Day 2” above remained outstanding at the start of the event proper. This is recorded because the project plan was originally dated from 20 September and therefore measured against the wrong calendar; it has since been re-anchored in `docs/PLAN.md`.

The practical consequence is favourable: the environment and a first evidenced baseline existed on day two of a four-week event, and the questions in Section 4 were already formulated before any mentor contact was possible.

Objective

Planning and documentation only. No conformance runs were executed, so no findings are added or amended in this entry.

Decisions taken

1. **Deliverable code location.** The conformance runner, plan configurations and orchestration live in our own repository. The `OpenIDConformanceTest` classes and their wiring go on branches of our forks of `inji-verify` and `inji-certify`, shaped so that each could be opened as an upstream pull request. Note that `api-test/` lives inside those two repositories, not in `mosip-functional-tests`, which carries only `apitest-commons`.
2. **Target specification treated as configuration.** Because Q1 has no channel for an answer other than the community forum, and may not receive one, the runner will take the plan name and variant set as configuration rather than hard-coding them. Either answer to Q1 then costs a configuration change rather than a rewrite.
3. **Benchmark semantics fixed provisionally as “regression from recorded baseline”** (option (a) in Q4), with an expected-failures file under version control. Given that the current honest baseline is a full set of failures at DCQL extraction, an absolute pass-rate gate would be meaningless.

Status at end of Day 2

No change to the findings catalogue. Modules 02–11, the ID2 verifier plan, and the resolution of F-05 remain outstanding and are scheduled within Week 1.

2.3 Day 3 — Wednesday, 24 September 2026

Objective

Build the Week 2 conformance runner, and before writing any of it, settle the two unknowns that `docs/WEEK2.md` identified as determining the week’s shape. Both were resolved by reading the

conformance suite's source in the local clone rather than by experiment.

Unknown 1 resolved — the verifier handoff needs no browser

The verifier plans are inverted relative to the issuer plans: the suite plays the wallet and waits passively for the verifier to initiate. Its web UI presents this as a box into which a tester pastes an `openid4vp://` URI, which is why every run so far has been manual. The open question was whether that paste has an API equivalent, because if it did not, the entire week would have had to be re-planned around a semi-automated flow.

It does. `AbstractVP1FinalVerifierTest.start()` reads:

```
getBrowser().requestUriInput(env.getString("authorization_endpoint"),
"Paste the openid4vp:// authorization request produced by the verifier under
test; its query string will be delivered to this test's authorization endpoint.");
```

and `handleHttp()` dispatches on `path.equals("authorize")`. The test exposes `authorization_endpoint` via `exposeEnvString`, readable through `GET /api/runner/{id}`. Delivering an authorisation request is therefore an ordinary HTTP GET to that endpoint carrying the verifier's query parameters. The paste box is a convenience wrapper over exactly that call.

Consequence. A fully unattended verifier run is achievable. This substantially answers Q6, which we had expected to need a mentor.

Unknown 2 resolved — the screenshot gate is configuration, not manual work

The happy-flow module requires a screenshot of the verifier's verification-result page before reaching REVIEW. `AbstractVP1FinalVerifierTest` carries a purpose-built path for automated runs: `handleVerificationEvidenceRequest()` serves an HTML stand-in page, and `fillScreenshotPlaceholderViaBrowser` fills the placeholder when the test configuration contains a `browser` automation entry matching the evidence URL. The source comment is explicit that this exists because “in automated runs there is no verifier UI a human could take a real screenshot of”.

A `browser` block has been added to `configs/runner.json` accordingly. **This is unconfirmed against a live run** — the match pattern in particular is a reasonable guess at the evidence URL and must be validated before it is relied upon.

Work completed

- `runner/` package: configuration loading, a conformance suite API client, an Inji Verify client, the handoff, result normalisation, the benchmark gate, an orchestrator and a CLI entry point.
- `run-conformance.sh` providing `-component verify | -component certify | -combined`, as the problem statement requires.
- 24 unit tests, all passing, covering normalisation against the real Day 1 log and the gate's behaviour — including its failure path, since a gate that cannot fail is decoration.

Defect found in our own code

The gate initially set `regression` and `improvement` only on the branches that made them true, relying on the normalisation layer having pre-populated the defaults. Applied to any module assembled by another path this raised `KeyError` rather than reporting “no regression”. Caught by `test_gate.py` before any live run and fixed by setting both unconditionally. Recorded because it is the argument for writing the gate's failure-path tests first: the bug lived in the success path and only the absent-key case exposed it.

Status at end of Day 3

Runner complete and unit-tested, but **not yet executed against a live conformance suite** — Docker was not running on the development machine during this session. Nothing in this entry should be read as evidence that the harness works end to end. The first live run is the next task, and until it happens the handoff and screenshot mechanisms are source-derived expectations rather than confirmed behaviour.

No new findings against Inji Verify; no conformance modules were executed.

2.4 Day 4 — Monday, 29 September 2026

Objective

First execution of the harness against a live conformance suite. The Day 3 runner had been unit-tested only, and every claim about the handoff was source-derived. Today's question was whether the automated run measures the same thing as the manual Day 1 run — not merely whether it completes.

Environment

Conformance suite 5.3.1 (prebuilt images), Inji Verify 0.18.2, ngrok 3.39.11 on the static domain. Both uncommitted local modifications (the ngrok host in `docker-compose.yml`, `pre_registered` for *Mock Identity (SD JWT)*) confirmed intact after five days. Same variant as Day 1.

Runs executed

Test ID	Outcome	Notes
<i>(none)</i>	HARNESS ERROR	Run 1. Suite side worked; Inji Verify returned HTTP 400 to <code>/vp-session-request</code> . Our defect, not Verify's — see below. No module result.
hcuWEvxGbvRgt7e	FAILED / INTER- RUPTED	Run 2, <code>nonceMode=sdk</code> . First fully unattended run. Reproduces the Day 1 manual baseline exactly.
Q9MaDZyfpJhjiDp	FAILED / INTER- RUPTED	Run 3, <code>nonceMode=service</code> . Controlled experiment; differs from run 2 in the nonce source only.

Result 1 — the automated run reproduces the manual baseline exactly

Run 2 and the Day 1 manual baseline (`MtzipWoD2dVgtic0`) produced identical counts — 15 success, 4 failure, 3 warning, 7 info — and the identical set of seven non-success checks, terminating at the same place, `ExtractDCQLQueryFromAuthorizationRequest`. No check appears in one run and not the other. The handoff delivered with the full 300 seconds remaining, and the suite answered its `/authorize` endpoint with HTTP 302.

This is the result that validates the harness. Completing is not enough; a harness that completed while measuring something different from what a tester would observe by hand would be worse than none. The Day 3 expectation that delivery is an ordinary HTTP GET to the exposed `authorization_endpoint` is now confirmed behaviour, not inference.

Result 2 — controlled experiment confirms the F-01/F-02 root cause

On Day 2 the weak nonce was traced to `btoa(Date.now())` in the SDK, with `verify-service`'s own `SecureRandom` generator reached only as a fallback. That was a reading of the source. Runs 2 and 3 test it: they differ in one variable — whether the harness supplies the SDK-shaped nonce or lets `verify-service` generate one.

Check	sdk (run 2)	service (run 3)
VP1FinalEnsureMinimumNonceEntropy	WARNING, 71.68 bits	SUCCESS
CheckForInvalidCharsInNonce	FAILURE (=)	SUCCESS
Nonce sent	MTc5MDY5NjYONTQzNw==	78cb380cda818ebc189b80704e6f17a3
Counts (success / failure / warning)	15 / 4 / 3	17 / 3 / 2

Exactly the two nonce checks move; every other check is unchanged and both runs still terminate at DCQL extraction. F-01 and F-02 are therefore confirmed by three independent routes: the suite flags them, the source locates them in the SDK, and removing only the SDK's nonce clears them. The service-generated nonce is 32 hexadecimal characters — the 16 `SecureRandom` bytes of `SecurityUtils.generateNonce()`, as the source predicted. This materially strengthens the upstream report.

Measurement decision recorded: the nonce source

Driving `verify-service` directly rather than through the UI changes what is measured. If the harness sends no nonce, the service generates a sound one and the nonce findings silently vanish — not because anything was fixed, but because the harness bypassed the component that causes them. The runner therefore makes the nonce source explicit (`nonceMode`), defaults to `sdk` so that runs measure what the shipped product sends a wallet, and records the mode on every module's output. The SDK-shaped nonce is a deliberate, documented reproduction of the SDK line, verified against the three Day 1 nonces by a unit test.

Defects found in our own code

Four, all fixed today. Recorded because each would have produced wrong evidence.

1. **The harness sent no presentation definition.** `VPRequestCreateDto` takes it inline. Found by reading the DTO before the first run.
2. **The harness sent the raw config definition, which has no id.** `VPDefinitionResponseDto` requires one, so `verify-service` answered with a bare 400 (run 1). The UI wraps selected credentials in an envelope in `vpVerificationState.ts` (lines 58–63, 82–83): a fixed id, a fixed purpose, and *only* the credentials' `input_descriptors`. The harness now builds the same envelope.
3. **The gate passed a run that measured nothing.** Run 1's harness error had no result; the severity ordering read that as FAILED, equal to the FAILED baseline, and the gate reported PASSED. A harness error now always fails the gate while the module itself stays SKIP, so the fault is never recorded against MOSIP. *This is the most serious of the four*: with the current baseline, the harness could have broken completely and CI would have stayed green.
4. **A refactor deleted an exception class**, breaking the CLI while all unit tests passed — none imported the modules that used it. An import test now covers every runner module.

Observation — not a finding

The presentation definition id is a hard-coded constant in `vpVerificationState.ts:59`: every request from every deployment of this UI carries `c4822b58-7fb4-454e-b827-f8758fe27f9a`. Presentation Exchange describes the id as identifying the definition, and a constant is arguably fine for an unchanging definition — but here the definition's contents vary with which credentials are selected while the id does not. Not catalogued: we have not established that anything relies on the id being unique, and without a consequence it is not a defect. Raised as a question instead.

Status at end of Day 4

The harness runs unattended end to end and reproduces the manual baseline check-for-check. Carried-forward Week 1 items remaining: modules 02–11 and the ID2 verifier plan (now a single command each), and F-05, which today’s runs cannot settle — the harness forwards `client_metadata` only when Verify supplies it, and in the `pre_registered` path it does not, so the open question of whether the UI adds it at QR-build time still needs the UI’s QR payload. The screenshot mechanism is not yet exercised: both runs terminated at DCQL before reaching it.

Automated logs are archived unsigned under `logs/2026-09-29/`; anything reported upstream still needs the suite’s signed export.

3 Findings Catalogue

Severity reflects impact on certifiability and on security, not on our convenience.

F-01 — Nonce has insufficient entropy

HIGH **CONFIRMED**

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.2

Observation. The nonce in Verify’s OID4VP authorisation request is a base64-encoded millisecond Unix timestamp rather than a cryptographically random value.

Evidence — our analysis. Three samples decoded, across both request-construction paths:

clientIdScheme	nonce (base64)	decoded	Δ to issuedAt
did	MTc4OTg4MzIxMzQ5MA==	1789883213490	—
pre_registered	MTc4OTg4NDUzNjY3OQ==	1789884536679	120 ms
pre_registered	MTc4OTg4NDY1NjQzNg==	1789884656436	158 ms

Evidence — conformance suite, independently.

VP1FinalEnsureMinimumNonceEntropy — WARNING

“Nonce does not appear to have sufficient entropy (calculated Shannon entropy is below the threshold). Section 5.2 of OID4VP 1.0 Final requires the nonce to be a ‘fresh, cryptographically random number with sufficient entropy’; OpenID4VP PR #722 recommends at least 128 bits.”
actual: 73.68 expected: 96.0

Root cause — located 23 September, and it corrects our Day 1 attribution. The weak nonce is generated by the *SDK*, not by the service. In `inji-verify-sdk/src/utis/api.ts` lines 10–12:

```
const generateNonce = (): string => {
  return btoa(Date.now().toString());
};
```

used at `api.ts:89` and `api.ts:159`. `verify-ui` depends on `@injistack/react-inji-verify-sdk` 0.18.1, so this is the nonce that reaches the authorisation request. `btoa` is standard base64, which pads with = — the same line therefore causes F-02.

The service already implements this correctly. `verify-service/.../utis/SecurityUtils.java` generates a nonce from `SecureRandom` over 16 bytes. But at `VerifiablePresentationRequestServiceImpl.java` it is only a fallback: `vpRequestCreate.getNonce() != null ? vpRequestCreate.getNonce() : SecurityUtils.generateNonce()`. Because the SDK always supplies a nonce, the sound generator never executes in the normal flow.

This reframes the finding materially: it is *not* a missing capability but a correct implementation overridden by its caller, which makes it a small, well-scoped fix rather than a design change — and a much more actionable report for a maintainer. Our Day 1 write-up attributed it to Inji Verify generally, which was imprecise. Recorded as a reminder that the “read the source, don’t guess” rule applies to root causes as much as to behaviour.

Confirmed experimentally — 29 September. Two automated runs differing only in the nonce source (`hcuWEvxGbvRgt7e`, SDK-shaped; `Q9MaDZyfpJhjiDp`, service-generated): this check and F-02’s move from WARNING/FAILURE to SUCCESS, and nothing else changes. See the Day 4 entry.

Impact. The nonce exists to bind a wallet’s presentation to a specific request and prevent replay. A value derived from the current clock is predictable to within milliseconds, substantially weakening that guarantee. In a national identity deployment this is a meaningful security property, not a cosmetic issue.

Proposed action. File upstream against `mosip/inji-verify` as a single issue covering F-01 and F-02 together, since they share one root cause and one fix. Recommend `crypto.getRandomValues`

over 32 bytes, base64url-encoded without padding. Draft prepared at docs/upstream/ISSUE-01-nonce-entropy. not yet filed.

F-02 — Nonce contains non-URL-safe characters MEDIUM CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.2

Evidence.

CheckForInvalidCharsInNonce — FAILURE

“Non URL safe characters found in nonce. This may introduce interoperability issues.”

invalid_chars: ["="]

Observation. The nonce uses standard base64 with = padding rather than base64url. The = character is not URL-safe and requires percent-encoding when the nonce travels in a query string, which not all wallets handle consistently.

Root cause. Identical to F-01 — btoa in inji-verify-sdk/src/utils/api.ts:11 emits standard base64 with = padding rather than base64url.

Impact. Interoperability risk with third-party wallets. Same root cause as F-01 and fixed by the same change; the two are filed upstream as one issue.

F-03 — No DCQL support; test plan cannot proceed HIGH CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §6

Evidence — conformance suite.

ExtractDCQLQueryFromAuthorizationRequest — FAILURE

“dcql_query not found in authorization request parameters”

Evidence — source. A case-insensitive search for dcql across verify-service/src/main returns zero matches. Credential selection is implemented entirely via presentation_definition with input_descriptors (DIF Presentation Exchange), set in VerifiablePresentationRequestServiceImpl as the presentation_definition_uri claim.

Impact. This terminates the test. Because every module in the 1.0 Final verifier plan requires a DCQL query, **Inji Verify 0.18.2 cannot pass any test in that plan.** This is the single most consequential finding of Day 1 and is not fixable by configuration.

Open question for mentors. See Q1.

F-04 — Uses superseded presentation_definition MEDIUM CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final

Evidence.

CheckNoPresentationDefinitionInVpAuthorizationRequest — WARNING

“Authorization request contains legacy ‘presentation_definition’. VP1 Final uses ‘dcql_query’ for credential selection, so sending both is unnecessary and may indicate draft and final request formats are being mixed.”

Relationship to F-03. Two faces of the same gap: Verify implements the earlier draft’s query model. Recorded separately because a maintainer may address them independently (e.g. support both during a transition period).

F-05 — client_metadata absent in inline mode MEDIUM UNVERIFIED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.1; Annex B.2.2, B.3.4

Evidence.

VP1FinalValidateVpFormatsSupportedInClientMetadata — FAILURE

“client_metadata is missing or not a JSON object”

VP1FinalCheckForUnexpectedParametersInVpClientMetadata — WARNING

“client_metadata not present in request but it is mandatory to include vp_formats in it”

Why this is not yet confirmed. In the `did` (by-reference) path, Verify’s signed request object `did` contain `client_metadata` with a full `vp_formats` block advertising `ldp_vp` and `vc+sd-jwt`. In the `pre_registered` (inline) path, the `/vp-session-request` API response contained no `client_metadata` field, so the URI we constructed could not include one. It is not yet established whether Verify omits it in inline mode, or whether the UI adds it when building the QR code and our reconstruction simply dropped it.

Action required before reporting. Inspect the QR payload the Verify UI actually produces in `pre_registered` mode and compare against the API response.

F-06 — No `direct_post.jwt`; HAIP certification impossible CONFIRMED

HIGH

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final / HAIP

Evidence — source.

```
verify-service/src/main/java/io/inji/verify/shared/Constants.java
public static final String RESPONSE_MODE = "direct_post";
```

This is a `static final` constant, not a configuration property or environment variable. No code path produces the encrypted `direct_post.jwt` variant.

Evidence — conformance suite. The HAIP verifier plan (*OpenID for Verifiable Presentations 1.0 Final/HAIP: Test a verifier*) offers only `direct_post.jwt` in its Response Mode variant.

Impact. Inji Verify cannot be submitted for HAIP self-certification in its current state. HAIP is the profile relevant to high-assurance national identity deployments, which is MOSIP’s own domain.

Open question for mentors. See Q2.

F-07 — DID document verification method identifier inconsistency CONFIRMED

MEDIUM

Component: Inji Verify 0.18.2 **Spec:** W3C DID Core

Evidence. Retrieved from `https://<host>/v1/verify/did.json`:

```
"id": "did:web:<host>:v1:verify" (document subject)
"verificationMethod[0].id": "did:web:<host>:verify#key-0" (no :v1:)
"assertionMethod[0]": "did:web:<host>:verify#key-0" (no :v1:)
```

Origin. The default Docker Compose environment variables differ: `INJI_DID_VERIFY_URI` includes the `v1` path segment while `INJI_DID_VERIFY_PUBLIC_KEY_URI` does not.

Impact. Under `did:web` resolution rules, the key’s identifier resolves to `https://<host>/verify/did.json`, a path that does not exist on the deployment. A strict relying party resolving the verification method by its own identifier would fail.

F-08 — No `mdoc` / ISO 18013-5 support

LOW CONFIRMED

Component: Inji Verify 0.18.2

Evidence. Searches for `mso_mdoc`, `iso_md1` and `mdoc` across `verify-service/src/main` return zero matches.

Impact. The `iso_md1` credential format variant of the verifier plan cannot be exercised. Recorded for completeness; likely a deliberate scope decision rather than a defect.

**F-09 — Client identifier prefix has no overlap with 1.0 Final
CONFIRMED****MEDIUM****Component:** Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.9.2**Evidence.**`EnsureClientIdMatchesResponseUri` — FAILURE“For `redirect_uri` `client_id` prefix, the `client_id` (without prefix) must equal `response_uri`”`client_id:` "inji-verify-ui" `response_uri:` "https://<host>/v1/verify/vp-submission/direct-post"

Observation. Inji Verify supports two client identifier schemes, `did` and `pre_registered`. The 1.0 Final verifier plan offers three prefixes: `redirect_uri`, `x509_san_dns`, `x509_hash`. Verify has no X.509 material configured, so the two X.509 options are unreachable, and `redirect_uri` requires a client identifier Verify does not produce. **No variant combination in the plan matches Verify’s client identification model.**

Caveat. This failure is partly a consequence of the variant we selected. The reportable observation is the absence of overlap, not that any single value is “wrong”.

Open question for mentors. See Q3.

4 Open Questions for Mentors

These are the questions we would like to put to MOSIP mentors. Each is phrased to be answerable without our setup in front of them.

Note on channel. The kickoff seminar and mentor AMA scheduled for the start of the event did not take place, so the community forum is currently the only route to an answer, and no answer is guaranteed. Where a question shapes the build — Q1 above all — our approach is to make the affected choice a configuration value and to state the assumption plainly in the submission, rather than to block on a reply.

Q1 — Target specification version.

Inji Verify 0.18.2 implements OID4VP credential selection via `presentation_definition` (Presentation Exchange), whereas OID4VP 1.0 Final requires `dcql_query`. Consequently no test in the 1.0 Final verifier plan can complete. Which specification version should our harness target as the benchmark: 1.0 Final (accepting that the current baseline is a full set of failures), or the ID2 draft that the shipped implementation appears to target? Is DCQL support on the Inji roadmap, and if so for which release?

Q2 — HAIP certification intent.

The problem statement asks us to document “how to submit for self-certification”. The HAIP verifier plan mandates `direct_post.jwt`, but `RESPONSE_MODE` is a `static final` constant set to `direct_post` in `Constants.java`. Is encrypted response mode planned? Should our harness include the HAIP plan as a known-failing suite to track progress toward certification, or exclude it until the capability exists?

Q3 — Client identifier scheme.

Verify supports `did` and `pre_registered` client identifier schemes; the 1.0 Final plan offers `redirect_uri`, `x509_san_dns` and `x509_hash`. Is there a supported configuration that allows an Inji Verify deployment to present a client identifier the 1.0 Final conformance plan can validate, or is this a known gap?

Q4 — Benchmark semantics for `api-testrig`.

The problem statement asks for results “gated against a conformance benchmark”. Given that the honest current baseline is “all modules fail at DCQL extraction”, should the benchmark be (a) a frozen snapshot of today’s pass/fail set, so the gate catches regressions only, or (b) an aspirational target that the build fails against until conformance improves? Our current design assumes (a), with an expected-failures file under version control.

Q5 — Sample configuration shipped with Inji Verify.

The default `config.json` ships five credential types, four of which use `ldp_vc` with `clientIdScheme: did`. That combination cannot be exercised by the 1.0 Final plan at all. Is the shipped sample configuration intended to be certification-representative, or purely a demonstration fixture? This affects whether we report the configuration itself as a finding.

Q6 — Interactive tests and automation.

Every module in the verifier plan is interactive: it waits for a human to trigger a flow in the verifier and paste the resulting authorisation request, and the happy-flow module also requires a screenshot upload before it reaches REVIEW status. Our harness can drive request generation via Verify’s own API, but the screenshot-upload requirement appears to need `POST /api/log/{id}/images`. Is a fully unattended run of the verifier plan considered achievable, or is a documented semi-automated flow the expected deliverable?

Substantially answered by our own investigation on Day 3, and retained here only for confirmation. The authorisation request is delivered by an HTTP GET to the test’s exposed `authorization_endpoint`, and the screenshot placeholder is filled by the suite’s own `browser`

automation against an HTML stand-in page it serves itself. Both are source-derived and await confirmation against a live run.

Q7 — Scope of Inji Certify work.

The `docker-compose-injistack` setup requires a PKCS12 keystore obtained through Mimoto onboarding, a publicly resolvable DID endpoint, and an external authorisation server. For a two-person team on a four-week timeline, is there a lighter-weight Certify configuration suitable for conformance testing, or is the Collab sandbox the intended target for the issuer plan?

Q8 — Presentation definition id (raised 29 September).

`verify-ui` sends every presentation definition with the same hard-coded id (`vpVerificationState.ts:59`), while the definition's contents vary with the credentials selected. Is anything on the Inji side — or in Inji Wallet — expected to rely on that id distinguishing definitions? We have not treated this as a defect, because we have found no consequence of it.

5 Reproduction

To reproduce the Day 1 baseline run:

1. Follow `docs/SETUP.md` to install prerequisites and clone the repositories.
2. Follow `docs/RUNBOOK.md` to start the conformance suite, Inji Verify, and ngrok.
3. In `inji-verify/docker-compose/config/config.json`, ensure the *Mock Identity (SD JWT)* entry has `"clientIdScheme":"pre_registered"`.
4. In the conformance suite, create the plan *OpenID for Verifiable Presentations 1.0 Final: Test a verifier — alpha tests* with the variant recorded in Section 2.
5. Run module 01, `oid4vp-1final-verifier-happy-flow`.
6. In the Verify UI, request the *Mock Identity (SD JWT)* credential; capture the `/v1/verify/vp-session-requ` response.
7. Run `python3 scripts/build-oid4vp-uri.py` on that response and paste the resulting URI into the suite. The request expires 300 seconds after issue.
8. Export the signed log and store it under `logs/<date>/.`

Since Day 4 the whole reproduction above is one command: `./run-conformance.sh -component verify`. Findings from modules 02–11 and the ID2 verifier plan to be appended as they are executed.